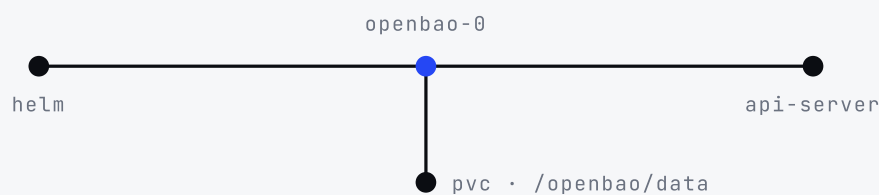


OPENBAO AUF KUBERNETES

Ein Secrets-Store im Cluster: Helm, Raft, init und unseal,
Kubernetes-Auth – und was danach noch fehlt.



OpenBao auf Kubernetes

Ein Secrets-Store im Cluster: Helm, Raft, init und unseal, Kubernetes-Auth – und was danach noch fehlt

Thomas Zachmann

17. September 2026

Inhalt

Worum es geht

- Warum von Hand, und warum ohne KI
- Ein Wort zu den Werten in diesem Leitfaden
- Lizenz und Haftung
- Für wen
- Die Bausteine
- Das Repository
- Der Cluster
- Auf kind statt RKE2

Teil I – Helm

- Das Chart
- Warum ein Pod und nicht drei
- Die Values
- Installieren
- Was `helm upgrade` später *nicht* tut

Teil II – init und unseal

- Warum das nicht automatisiert wird
- init
- unseal
- Was nach jedem Neustart passiert
- Der erste Login

Teil III – Kubernetes-Auth

- Wie es funktioniert
- Aktivieren und konfigurieren
- Eine Rolle für einen Consumer
- Prüfen, ohne ESO

Teil IV – Der Zugang von außen

- Der Weg
- Der `bao-CLI` von außen
- Die UI

Teil V – Der Ist-Zustand, ehrlich

Teil VI – Härtung: was als Nächstes kommt

1. Token-Datei aus dem Pod entfernen
2. Audit-Device
3. Admin-Policy und userpass
4. Den Ersatzzugang beweisen – nicht überspringen
5. Root-Token widerrufen
6. Ein erster Snapshot

Teil VII – Was schiefgehen kann

Pod bleibt 0/1 Running

Log voller service_registration ... 403

helm upgrade ändert nichts

403 permission denied bei allem

Kubernetes-Login schlägt fehl

OpenBao nimmt keine Anfragen mehr an, Log sagt audit

generate-root antwortet 405 unsupported operation

Nach Node-Wartung: versiegelt

Anhang A – Alle Kommandos

Anhang B – Glossar

Anhang C – Der Reverse-Proxy

Warum DuckDNS, und warum DNS-01

1. DuckDNS

2. Die Proxy-VM

3. Das Zertifikat

4. Der Proxy Host

Prüfen

Was hier fehlt, ehrlich

Worum es geht

Kubernetes bringt ein eigenes Objekt für Geheimnisse mit, das `Secret`. Es speichert Passwörter und Schlüssel, mehr nicht: Es protokolliert nicht, wer sie liest, es erzeugt keine Zugänge mit Ablaufdatum, und es unterscheidet nicht, welcher Workload danach fragt. Ein Cluster braucht darum einen Ort für Geheimnisse, der diese drei Dinge kann. Dieser Leitfaden baut genau das: **OpenBao im Cluster**, als `StatefulSet` mit `Raft-Storage`, per Helm installiert, manuell initialisiert und entsiegelt, mit Kubernetes-Auth als Login-Methode für Workloads.

Es ist der erste Teil einer Reihe. Der zweite, *Dynamische Datenbank-Credentials mit OpenBao*, setzt genau auf diesem Aufbau auf und lässt eine Anwendung kurzlebige PostgreSQL-Zugänge daraus beziehen. Der dritte beschreibt denselben OpenBao auf einer virtuellen Maschine – mit Ansible, OpenTofu und einem geprüften Disaster Recovery. Viele Entscheidungen in diesem Leitfaden sind von dort übernommen, und wo der Cluster-Aufbau *hinter* dem VM-Aufbau zurückbleibt, steht das ausdrücklich dabei.

Alles hier wurde tatsächlich auf einem RKE2-Cluster im Lab durchgeführt. Der Ist-Zustand am Ende ist ehrlich beschrieben – inklusive dessen, was noch nicht gemacht ist.

Warum von Hand, und warum ohne KI

In einem echten Setup wird ein Helm-Release nicht per Hand deployt, sondern per GitOps, und die OpenBao-Konfiguration nicht per CLI, sondern per OpenTofu. Trotzdem geht dieser Leitfaden jeden Schritt einmal von Hand – und bewusst ohne KI-Assistenz als Abkürzung. Wer ein Sprachmodell die Values schreiben lässt, bekommt ein laufendes OpenBao und keine Ahnung, warum `tlsDisable` gesetzt ist, was `Raft` von `file` unterscheidet oder warum der Pod nach jedem Neustart versiegelt ist.

Der Maßstab: Wer diesen Leitfaden durchgearbeitet hat, kann auf einem leeren Blatt zeichnen, wie ein Pod im Cluster an ein `Secret` aus OpenBao kommt – über welche Objekte, mit welchem Token, geprüft von wem. Wer das kann, kann auch das Helm-Release in Flux oder Argo einhängen und die Konfiguration nach OpenTofu übertragen.

Ein Wort zu den Werten in diesem Leitfaden

Der Cluster ist eine Entwicklungsumgebung: kein Zugang von außen, keine echten Daten. Hostnamen und interne Adressen sind durch Platzhalter ersetzt (`bao.example.internal`). `Unseal-Keys` und `Root-Token` erscheinen nirgends – sie stehen auch im Original nicht im Terminal-Log, weil `init` bewusst nie in einer Automatisierung oder einem Chat-Fenster läuft.

Lizenz und Haftung

Dieser Leitfaden steht unter CC BY 4.0: Er darf kopiert, weitergegeben und bearbeitet werden, auch kommerziell, solange der Autor genannt wird. Er wird ohne Gewähr bereitgestellt. Alles darin wurde in einer Entwicklungsumgebung durchgeführt; wer es in einer anderen Umgebung nachvollzieht, tut das auf eigene Verantwortung.

Für wen

Für Leserinnen und Leser, die Kubernetes-Grundlagen kennen (StatefulSet, PVC, Ingress, ServiceAccount, Helm) und wissen, was ein Secrets-Manager ist. OpenBao ist ein Fork von HashiCorp Vault; alles hier gilt für beide, nur der CLI-Name (bao) und die Umgebungsvariablen (BAO_ADDR) unterscheiden sich. Das Container-Image macht den Übergang noch weicher: `/usr/bin/vault` ist darin ein Symlink auf `/usr/bin/bao`, `vault status` in der Pod-Shell tut also dasselbe wie `bao status` – und `vault version` antwortet mit `OpenBao v2.6.2`. Alte Skripte und Gewohnheiten laufen weiter; dieser Leitfaden schreibt trotzdem durchgehend `bao`.

Wer die Konzepte – Seal/Unseal, Auth-Methoden, Policies, Secrets Engines, Leases – von Grund auf verstehen will, mit Labs, die auf dem Laptop laufen, findet das in meinem Buch **Vault in Practice** (Leanpub, leanpub.com/vault-in-practice). Dieser Leitfaden setzt diese Konzepte voraus und zeigt, wie sie im Cluster zusammenkommen.

Die Bausteine

Baustein	Version	Rolle
Ubuntu Server	24.04 LTS	das Betriebssystem der sechs Nodes
RKE2 Kubernetes	v1.36.4+rke2r1	3 Control-Plane-Nodes, 3 Worker, Cilium als CNI, Traefik als Ingress, Longhorn als Storage
OpenBao Helm-Chart openbao/openbao	Chart 0.29.4, App 2.6.2	Das Deployment
Helm	3.16	Installiert das Chart
bao CLI	2.6.2	Im Pod enthalten; lokal optional

Das Repository

Die Konfigurationsdateien für den Cluster (`rke2/`) und die Values-Datei, die Teil I Stück für Stück erklärt (`k8s/values.yaml`), stehen nicht vollständig im Text, sondern im Repository zu dieser Reihe. Der Grund ist das PDF: Shell-Kommandos lassen sich daraus kopieren, YAML und Policies nicht – führende Leerzeichen sind im PDF keine Zeichen, sondern nur Position, und beim Kopieren sind sie weg. Alles, was von Einrückung lebt, kommt darum aus dem Repository. Alle relativen Pfade in diesem Leitfaden meinen das Verzeichnis dieser Note. Also zuerst klonen und hineinwechseln:

```
git clone https://github.com/thomaszachmann/field-notes.git
cd field-notes/openbao/01-openbao-on-kubernetes
ls rke2/ k8s/ openbao/ commands.sh
```

Von hier aus laufen alle `scp`-, `helm`- und `kubectl`-Aufrufe der folgenden Teile. Und wer auch die Kommandos nicht aus dem PDF kopieren will (manche Viewer zerlegen Wörter mit Unterstrichen beim Kopieren): `commands.sh` ist Anhang A als Datei.

Der Cluster

Der Leitfaden setzt einen laufenden Kubernetes-Cluster voraus. Hier ist es RKE2, die Kubernetes-Distribution von Rancher: sechs Nodes mit Ubuntu Server 24.04 LTS auf einem physischen Host, drei davon Control-Plane (RKE2 nennt sie *Server*), drei Worker (*Agents*). Wer schon einen Cluster mit einer Ingress-Klasse und einer StorageClass hat, kann diesen Abschnitt überspringen und später in den Values `ingressClassName` und `storageClass` anpassen.

Der Aufbau in Kurzform, damit klar ist, wo der Cluster herkommt – die vollständige Anleitung steht unter docs.rke2.io/install/ha. Alle Kommandos laufen als root auf der jeweiligen Maschine.

1. Der erste Server. Die Konfiguration steht in `/etc/rancher/rke2/config.yaml`, der Installer liest sie beim Start. Die Datei liegt als `rke2/config-server-1.yaml` im Repository:

```
token: <cluster-token>
cni: cilium
node-taint:
  - "CriticalAddonsOnly=true:NoExecute"
```

Vom Arbeitsrechner auf den Server kopieren, dort den Token eintragen, installieren:

```
ssh root@<erster-server> mkdir -p /etc/rancher/rke2
scp rke2/config-server-1.yaml root@<erster-server>:/etc/rancher/rke2/config.yaml
ssh root@<erster-server>
# auf dem Server: Platzhalter durch den Token ersetzen
vi /etc/rancher/rke2/config.yaml
curl -sL https://get.rke2.io | sh -
systemctl enable --now rke2-server.service
```

token ist das Geheimnis, mit dem sich alle weiteren Nodes anmelden – frei gewählt, z. B. mit `openssl rand -hex 32` erzeugt, und wie die Unseal-Keys später in den Passwort-Manager. Lässt man es weg, erzeugt RKE2 selbst eines und legt es unter `/var/lib/rancher/rke2/server/node-token` ab. Einen festen Namen für den API-Server (`tls-san`) braucht es erst, wenn eine VIP oder ein DNS-Name vor den drei Servern stehen soll; im Lab reicht die IP des ersten Servers, die ohnehin im Zertifikat steht. Der `node-taint` hält Workloads von den Control-Plane-Nodes fern – und ist der Grund, warum Traefik in Teil IV nur auf den Workern läuft.

`cni: cilium` wählt das Netzwerk-Plugin. Ohne die Zeile nimmt RKE2 Canal; beides erzwingt `NetworkPolicy` (N° 7 verlässt sich darauf), Cilium tut es mit eBPF und kann später Hubble und den kube-proxy-Ersatz dazuschalten (docs.rke2.io/networking/basic_network_options). Die Wahl fällt vor dem ersten Start: Ein CNI-Wechsel im Betrieb ist kein Upgrade, sondern ein Neubau.

Der erste Start dauert einige Minuten, weil RKE2 seine Images lädt. `journalctl -u rke2-server -f` zeigt den Fortschritt; fertig ist er, wenn `/etc/rancher/rke2/rke2.yaml` existiert. Bricht die Unit sofort ab, steht der Grund in derselben Ausgabe – bei `yaml: line N: could not find expected ':'` ist die Konfigurationsdatei kaputt, meist durch verlorene Einrückung.

2. Die beiden anderen Server. Dieselbe Konfiguration, ergänzt um die Adresse des ersten Servers (`rke2/config-server-n.yaml`), dann derselbe Installer und dieselbe systemd-Unit:

```
server: https://<first-server-ip>:9345
token: <cluster-token>
cni: cilium
node-taint:
  - "CriticalAddonsOnly=true:NoExecute"
```

Drei Server ergeben ein etcd-Quorum: Einer darf ausfallen.

3. Die Worker. Nur server und token (rke2/config-agent.yaml) – CNI und Taints kommen von den Servern – und der Installer als Agent:

```
server: https://<first-server-ip>:9345
token: <cluster-token>
```

```
curl -sfL https://get.rke2.io | INSTALL_RKE2_TYPE="agent" sh -
systemctl enable --now rke2-agent.service
```

4. kubectl und helm auf dem Arbeitsrechner. Auf dem Server selbst liegt ein kubectl unter /var/lib/rancher/rke2/bin/, aber gearbeitet wird vom eigenen Rechner aus. kubectl darf höchstens eine Minor-Version vom Cluster abweichen (hier also 1.35 bis 1.37), helm ist 3.16 oder neuer:

```
# macOS
brew install kubectl helm

# Linux (amd64)
KUBECTL_VERSION=$(curl -Ls https://dl.k8s.io/release/stable.txt)
curl -LO "https://dl.k8s.io/release/$KUBECTL_VERSION/bin/linux/amd64/kubectl"
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
curl -fsSL https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 \
  | bash

kubectl version --client
helm version
```

Die offiziellen Anleitungen, auch für Windows und andere Architekturen:

kubernetes.io/docs/tasks/tools und helm.sh/docs/intro/install.

5. Die Kubeconfig. Der erste Server schreibt sie nach /etc/rancher/rke2/rke2.yaml, mit 127.0.0.1 als Adresse. Auf den eigenen Rechner kopieren und die Adresse durch die IP des ersten Servers ersetzen:

```
mkdir -p ~/.kube
scp root@<erster-server>:/etc/rancher/rke2/rke2.yaml ~/.kube/config
chmod 600 ~/.kube/config
kubectl config set-cluster default --server=https://<first-server-ip>:6443
kubectl get nodes
```

Alle sechs Nodes sollten Ready sein, die drei Server mit den Rollen control-plane, etcd, master. Und Cilium läuft als DaemonSet auf jedem Node:

```
kubectl get pods -n kube-system -l k8s-app=cilium
```

6. Ingress und Storage. Seit RKE2 v1.36 ist Traefik der Standard-Ingress-Controller; er kommt als DaemonSet auf den Workern mit, ohne weiteres Zutun. Longhorn kommt per Helm dazu – einmal, für den ganzen Cluster. Es läuft als DaemonSet auf den Nodes, die es zulassen: wegen des Taints aus Schritt 1 nur auf den drei Workern, und dort liegen auch die Replikate (Default: drei, eines pro Worker). Vorher auf diesen drei Nodes `open-iscsi`, weil Longhorn Volumes per iSCSI an den Node hängt. Auf Ubuntu 24.04:

```
# auf jedem Worker, als root
apt-get update && apt-get install -y open-iscsi
systemctl enable --now iscsid
```

Dann vom Arbeitsrechner:

```
helm repo add longhorn https://charts.longhorn.io
helm repo update
helm upgrade --install longhorn longhorn/longhorn \
  --namespace longhorn-system --create-namespace
kubectl get storageclass
```

Danach gibt es die StorageClass `longhorn`, die die Values in Teil I verlangen. Dass Longhorn dort gelandet ist, wo es hingehört:

```
# erwartet: drei Pods, je einer auf einem Worker
kubectl get pods -n longhorn-system -l app=longhorn-manager -o wide
# erwartet: die drei Worker, Schedulable true
kubectl get nodes.longhorn.io -n longhorn-system
```

7. Der Weg von außen. Alles bis hierher ist im Cluster. Ein Browser oder die `bao`-CLI auf dem Arbeitsrechner müssen aber von außen an OpenBao herankommen, und dafür fehlen drei Dinge: ein **Name**, unter dem OpenBao erreichbar ist, ein **Zertifikat** für diesen Namen, und eine **Stelle**, die Anfragen an diesen Namen in den Cluster weiterreicht.

Diese Stelle ist hier ein Reverse-Proxy auf einer eigenen kleinen VM vor dem Cluster. Er hält das Zertifikat und beendet TLS; alles hinter ihm spricht Klartext-HTTP. Das ist der Grund für `tlsDisable: true` in den Values von Teil I: OpenBao selbst sieht nie ein Zertifikat. Der Weg einer Anfrage:

```
Browser —https→ Reverse-Proxy —http→ Worker:80 (Traefik) → Ingress → svc/openbao:8200
```

Warum Port 80 eines **Workers**: Traefik läuft als DaemonSet mit `hostPort 80/443`, aber wegen des Taints aus Schritt 1 nur auf den Workern. Der Proxy zeigt also auf die IP eines Workers; fällt der aus, ist die UI weg, obwohl OpenBao weiterläuft. Was das bedeutet und warum das im Lab hingenommen wird, steht in Teil IV.

Traefik entscheidet am `Host-Header`, welcher Ingress gemeint ist. Der Name, den der Proxy weiterreicht, muss darum derselbe sein wie in `server.ingress.hosts` der Values – in diesem Leitfaden steht dafür der Platzhalter `bao.example.internal`. Wie Name, Zertifikat und Proxy hier konkret entstanden sind – Nginx Proxy Manager, DuckDNS, Let's Encrypt – steht Schritt für Schritt in Anhang C. Wer schon einen Proxy hat, trägt dort nur den Namen und die Worker-IP ein.

Ob der Weg bis Traefik steht, lässt sich schon jetzt prüfen, bevor OpenBao installiert ist:

```
curl -s -o /dev/null -w '%{http_code}\n' \
-H 'Host: bao.example.internal' http://<worker-ip>/
```

404 ist hier richtig: Traefik antwortet, kennt den Host aber noch nicht. Sobald der Ingress aus Teil I existiert, wird daraus die Antwort von OpenBao.

Auf kind statt RKE2

Wer keine sechs Maschinen hat, sondern einen Laptop mit Docker: Teil I bis VII laufen auch auf [kind](#) – Kubernetes in einem Docker-Container. Was fehlt, ist genau das, was oben eingerichtet wurde: Longhorn, Traefik, der Reverse-Proxy – und statt Cilium bringt kind sein eigenes CNI mit (kindnet, erzwingt seit 0.24 ebenfalls NetworkPolicy). Der Ersatz dafür, durchgeführt mit kind v0.32.0:

Ein Cluster mit einem Node, dessen Port 80 auf den Laptop gemappt ist, und ingress-nginx darauf – das ist der Weg, den die kind-Dokumentation für Ingress vorsieht:

```
cat > kind-config.yaml <<'EOF'
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
- role: control-plane
  extraPortMappings:
  - containerPort: 80
    hostPort: 80
EOF
kind create cluster --name bao --config kind-config.yaml
kubectl apply -f https://kind.sigs.k8s.io/examples/ingress/deploy-ingress-nginx.yaml
```

```
$ kubectl get nodes
NAME                STATUS    ROLES    AGE   VERSION
bao-control-plane   Ready    control-plane   2m13s   v1.36.1

$ kubectl get storageclass
NAME                PROVISIONER          RECLAIMPOLICY   VOLUMEBINDINGMODE
standard (default)  rancher.io/local-path Delete           WaitForFirstConsumer

$ kubectl get ingressclass
NAME    CONTROLLER          PARAMETERS
nginx   k8s.io/ingress-nginx <none>
```

Kein Longhorn: kind bringt standard (local-path) als Default-StorageClass mit. Kein Traefik, kein Proxy: ingress-nginx hört auf Port 80 des Node-Containers, und der liegt auf localhost. Als Hostname reicht bao.127.0.0.1.nip.io – nip.io löst jeden Namen mit einer IP darin auf genau diese IP auf, ohne DNS-Eintrag und ohne Anhang C.

Damit ändern sich in k8s/values.yaml drei Werte. Statt die Datei anzufassen, kommen sie in Teil I als Overrides an den helm-Aufruf:

```
helm upgrade --install openbao openbao/openbao \
  --version 0.29.4 \
  --namespace openbao --create-namespace \
  --values k8s/values.yaml \
  --set server.dataStorage.storageClass=standard \
  --set server.ingress.ingressClassName=nginx \
  --set 'server.ingress.hosts[0].host=bao.127.0.0.1.nip.io'
```

Danach sieht es aus wie in Teil I, nur mit anderen Namen in den Spalten CLASS und STORAGECLASS:

```
$ kubectl get sts,ingress,pvc -n openbao
NAME                                READY   AGE
statefulset.apps/openbao           0/1     73s
```

NAME	CLASS	HOSTS	ADDRESS	PORTS
ingress.networking.k8s.io/openbao	nginx	bao.127.0.0.1.nip.io	localhost	80

NAME	STATUS	CAPACITY	STORAGECLASS
persistentvolumeclaim/data-openbao-0	Bound	10Gi	standard

Und von außen, ohne TLS – der Listener spricht HTTP, und diesmal steht kein Proxy davor, der das ändert:

```
$ export BAO_ADDR=http://bao.127.0.0.1.nip.io
$ bao status
Key                Value
---                -
Seal Type          shamir
Initialized        false
Sealed             true
Version            2.6.2
Storage Type       raft
HA Enabled         true
```

Ab hier gilt Teil II bis VII wie gedruckt: init und unseal in der Pod-Shell, Kubernetes-Auth über TokenReview (das ClusterRoleBinding auf system:auth-delegator rendert das Chart auch hier), Snapshots per kubectl cp. Drei Dinge liest man anders: https://bao.example.internal ist überall http://bao.127.0.0.1.nip.io; „Worker“ in Teil IV meint auf kind den einen Node; und storageClass: longhorn beim Audit-Volume in Teil VI wird ebenfalls standard. Und: kind delete cluster --name bao nimmt das PVC mit. Snapshots sind auf dem Laptop nicht weniger Pflicht, nur schneller vergessen.

Teil I – Helm

Das Chart

OpenBao pflegt ein eigenes Chart, abgeleitet vom Vault-Chart. Es kann drei Betriebsarten: dev (In-Memory, unversiegelt – nur zum Spielen), standalone (ein Pod, persistentes Volume) und ha (mehrere Pods mit Raft-Quorum). Dieser Leitfaden nimmt standalone.

```
helm repo add openbao https://openbao.github.io/openbao-helm
helm repo update
helm search repo openbao/openbao --versions | head -3
```

Warum ein Pod und nicht drei

Drei Worker, Longhorn, ein Chart, das ha kann – die Frage liegt nahe. Was drei Pods bringen: Stirbt der aktive Pod oder sein Worker, übernimmt ein Standby in Sekunden, und Upgrades laufen Pod für Pod ohne Ausfall. Für alles, was dauerhaft an OpenBao hängt – der External Secrets Operator ab N° 2, cert-manager ab N° 5 – wäre das der eigentliche Gewinn. Was sie nicht bringen: Schutz vor dem Ausfall des einen physischen Hosts. Sechs VMs auf einer Maschine sind sechs Pods auf einer Maschine.

Und was sie kosten, solange die Unseal-Keys von Hand eingegeben werden (Teil II): Jeder Pod hat seinen eigenen Seal. Nach einem Stromausfall sind es neun Key-Eingaben statt drei – und bis alle drei entsiegelt sind, gibt es kein Quorum und damit gar kein OpenBao. Ein Standby, der nach einem Neustart versiegelt dasteht, übernimmt nichts. Mit Shamir wird ha also nicht robuster als standalone, sondern anfälliger. Dazu: drei PVCs, jedes von Longhorn dreifach repliziert, also neun Kopien derselben Daten; und der service_registration-Block, den dieser Leitfaden unten wegen seiner 403-Warnungen streicht, wird gebraucht, damit der Service den *aktiven* Pod findet.

ha lohnt sich, sobald zwei Dinge da sind: **Auto-Unseal**, damit sich alle drei Pods selbst entsiegeln (N° 6: Transit-Seal gegen den VM-OpenBao), und **etwas, das den Ausfall spürt**. Beides kommt später in der Reihe. Bis dahin ist ein Pod die ehrlichere Wahl – er ist genauso oft versiegelt, aber nur einmal.

Die Values

Die vollständige Datei (k8s/values.yaml) hat 70 Zeilen; hier die Teile mit Begründung.

```
global:
  tlsDisable: true
```

TLS wird am Reverse-Proxy terminiert. Das Chart nutzt diesen Schalter an mehreren Stellen: Es setzt BAO_ADDR im Pod auf http://, konfiguriert die Probes ohne TLS und lässt den tls-Block im Ingress weg. Setzt man nur den Listener auf tls_disable = 1, aber nicht global.tlsDisable, gehen die Readiness-Probes ins Leere.

```
injector:
  enabled: false
```

Am Ende soll eine Anwendung in einem Pod ein Secret aus OpenBao benutzen. Die Anwendung selbst weiß nichts von OpenBao – irgendetwas muss das Secret also holen und ihr hinlegen. Dafür gibt es zwei verbreitete Wege, und der Schalter entscheidet über den ersten.

Der **Injector** ist ein Hilfsprogramm, das das Chart optional mit installiert (es stammt aus dem Vault-Umfeld, Projekt `vault-k8s`). Es schaltet sich in den Moment ein, in dem Kubernetes einen neuen Pod anlegt: Trägt der Pod eine bestimmte Markierung (eine Annotation wie `vault.hashicorp.com/agent-inject: "true"`), baut der Injector den Pod unbemerkt um und setzt einen zweiten, kleinen Container daneben – einen *Sidecar*. Der meldet sich bei OpenBao an, holt das Secret und schreibt es als Datei in ein Verzeichnis, das beide Container sehen. Die Anwendung liest dann einfach eine Datei.

Der **External Secrets Operator** (ESO) geht anders vor: Er läuft einmal im Cluster, holt Secrets aus OpenBao und legt sie als ganz normale Kubernetes-Secrets ab. Die Anwendung bekommt sie wie jedes andere Secret – als Umgebungsvariable oder als Datei – und der Pod bleibt unverändert.

Dieser Leitfaden und die folgenden (N° 2, N° 4) nehmen den zweiten Weg. Darum bleibt der Injector aus: Er würde nur laufen, ohne gebraucht zu werden – und ein Programm, das jeden neuen Pod umbauen darf, ist eines, das man nicht ohne Grund im Cluster haben will.

```
server:
  standalone:
    enabled: true
    config: |
      ui = true

      listener "tcp" {
        address      = "[::]:8200"
        cluster_address = "[::]:8201"
        tls_disable   = 1
      }

      storage "raft" {
        path = "/openbao/data"
      }
```

Der wichtigste Eingriff ist `storage "raft"`. Der Chart-Default für `standalone` ist `storage "file"` – ein Backend, das **keine Snapshots kann**. `bao operator raft snapshot save` existiert nur mit Raft. Wer den Default lässt, hat keinen Weg, ein konsistentes Backup zu ziehen, außer das PVC zu kopieren – und das ist bei laufendem Prozess nicht konsistent.

`api_addr` und `cluster_addr` fehlen bewusst: Das Chart setzt `BA0_API_ADDR` und `BA0_CLUSTER_ADDR` als Umgebungsvariablen auf dem StatefulSet, auch im Standalone-Modus, und OpenBao liest sie beim Start.

`service_registration "kubernetes"` stand im ursprünglichen Setup ebenfalls in diesem Block – und ist in `k8s/values.yaml` bewusst **nicht** mehr enthalten. Die Registrierung soll den Zustand (`active`, `sealed`, ...) als Labels an den eigenen Pod schreiben, damit ein HA-Service nur den aktiven Knoten

selektiert. Im Standalone-Modus rendert das Chart aber weder die Role noch das RoleBinding dafür, und OpenBao protokolliert dann alle fünf Sekunden:

```
[WARN] service_registration.kubernetes: unable to set initial state due; will retry:
err="GET https://10.43.0.1:443/api/v1/namespaces/openbao/pods/openbao-0 ..."
resp statusCode: 403"
```

22 Stunden lang, bevor es jemandem auffiel. Die Services selektieren im Standalone-Modus ohnehin auf `component: server` mit `publishNotReadyAddresses: true` – die Labels würden nichts ändern. Also weglassen, oder eine Role mit `get/patch` auf pods ergänzen, wenn man die Labels für Monitoring will.

```
dataStorage:
  enabled: true
  size: 10Gi
  storageClass: longhorn

persistentVolumeClaimRetentionPolicy:
  whenDeleted: Retain
  whenScaled: Retain
```

Beides explizit, obwohl es die Defaults wären. Der Grund ist derselbe wie bei `ingressClassName`: Ein geänderter Cluster-Default darf die Raft-Daten nicht stillschweigend woanders hinlegen oder bei `helm uninstall` mitnehmen.

```
ingress:
  enabled: true
  ingressClassName: traefik
  hosts:
    - host: bao.example.internal
      paths: []

ui:
  enabled: true
  serviceType: ClusterIP
```

Kein `tls`-Block, kein `cert-manager`. Der Reverse-Proxy vor dem Cluster hat das Zertifikat und leitet per HTTP an Traefik weiter.

Installieren

```
helm upgrade --install openbao openbao/openbao \
  --version 0.29.4 \
  --namespace openbao --create-namespace \
  --values k8s/values.yaml
```

Ohne `--wait`. Die Readiness-Probe des Charts ist `bao status`, und die schlägt fehl, solange OpenBao versiegelt ist. Auf einer frischen Installation ist der Pod erst nach `init + unseal Ready` – `--wait` würde nur in den Timeout laufen.

Vorher prüfen, ohne etwas zu ändern:

```
helm upgrade --install openbao openbao/openbao --version 0.29.4 \
--namespace openbao --values k8s/values.yaml --dry-run=server
```

Danach:

```
$ kubectl get sts,svc,ingress,pvc -n openbao
```

NAME	READY	AGE
statefulset.apps/openbao	0/1	1m

NAME	TYPE	CLUSTER-IP	PORT(S)
service/openbao	ClusterIP	10.43.179.240	8200/TCP,8201/TCP
service/openbao-internal	ClusterIP	None	8200/TCP,8201/TCP
service/openbao-ui	ClusterIP	10.43.158.186	8200/TCP

NAME	CLASS	HOSTS	PORTS
ingress.networking.k8s.io/openbao	traefik	bao.example.internal	80

NAME	STATUS	CAPACITY	STORAGECLASS
persistentvolumeclaim/data-openbao-0	Bound	10Gi	longhorn

READY 0/1 ist hier richtig. Der Pod läuft, OpenBao antwortet, aber es ist nicht initialisiert – und das Chart nennt das zu Recht „nicht bereit“.

Was `helm upgrade` später *nicht* tut

Normalerweise gilt in Kubernetes: Ändert sich die Vorlage eines Pods – ein anderes Image, eine andere Umgebungsvariable, eine neue Konfiguration –, ersetzt Kubernetes den laufenden Pod von selbst durch einen neuen. Bei einem StatefulSet heißt das *Rolling Update*, und es passiert sofort nach dem `helm upgrade`, ohne Nachfrage.

Für OpenBao wäre das ein Problem. Jeder neue Pod startet **versiegelt** (Teil II): Er läuft, antwortet aber auf nichts, bis jemand drei Unseal-Keys eingibt. Ein Rolling Update um drei Uhr nachts wäre damit ein Ausfall bis zum Morgen. Darum stellt das Chart das StatefulSet auf `updateStrategyType: OnDelete`. Das bedeutet: Kubernetes merkt sich die neue Vorlage, rührt den laufenden Pod aber nicht an. Erst wenn man den Pod selbst löscht, entsteht der neue – mit der neuen Konfiguration, und versiegelt.

So sieht das aus, wenn man nach einem `helm upgrade` mit geänderten Values nachschaut:

```
$ kubectl get sts -n openbao openbao \
-o jsonpath='{.spec.template.spec.containers[0].resources.requests.cpu}'
150m
$ kubectl get pod -n openbao openbao-0 \
-o jsonpath='{.spec.containers[0].resources.requests.cpu}'
100m
```

Das StatefulSet hat den neuen Wert, der Pod noch den alten. Die Änderung wird wirksam, wenn man will – und nur dann:

```
kubectl -n openbao delete pod openbao-0      # Pod kommt neu, versiegelt
kubectl exec -it -n openbao openbao-0 -- bao operator unseal  # x3
```

Das ist derselbe Kompromiss wie ein Konfigurations-Neustart auf einer VM: Ein Neustart kostet ein Unseal, also passiert er nur absichtlich. Wer nach einem `helm upgrade` vergisst, den Pod zu löschen, läuft mit der alten Konfiguration weiter – und wundert sich, warum die Änderung nichts tut (Teil VII).

Teil II – init und unseal

Warum das nicht automatisiert wird

`bao operator init` erzeugt die Unseal-Keys und den Root-Token. Diese Ausgabe erscheint **genau einmal**. Läuft `init` in einer Pipeline, in Ansible oder in einem KI-Chat, steht das Key-Material in Logs, in Task-Ausgaben oder in einem Kontextfenster. Deshalb: ein normales Terminal, eine Shell im Pod, Ausgabe sofort in den Passwort-Manager, dann Terminal schließen.

init

```
kubectl exec -it -n openbao openbao-0 -- sh
/ $ bao operator init -key-shares=5 -key-threshold=3
```

Shamir 5 von 3: fünf Schlüssel, drei reichen zum Entsiegeln. Bei einem Lab, in dem eine Person alle fünf hält, ist das kein Sicherheitsgewinn gegenüber 1/1 – aber es ist das Format, das ein späterer Wechsel zu mehreren Verwahrern erwartet, und es kostet nichts.

Die Ausgabe enthält `Unseal Key 1` bis `5` und `Initial Root Token`. Alles in den Passwort-Manager. **Nicht** in dieses Repo, **nicht** auf das NAS, auf dem später die Snapshots liegen – die Snapshots sind mit genau diesen Keys verschlüsselt, und wer beides an einem Ort hat, hat beides verloren, wenn der Ort weg ist.

unseal

```
/ $ bao operator unseal      # Key eingeben, ohne Echo
/ $ bao operator unseal
/ $ bao operator unseal
/ $ bao status
```

Key	Value
---	-----
Seal Type	shamir
Initialized	true
Sealed	false
Total Shares	5
Threshold	3
Version	2.6.2
Storage Type	raft
HA Enabled	true
HA Cluster	https://openbao-0.openbao-internal:8201
HA Mode	active

`HA Enabled true` bei einem einzelnen Pod überrascht – es ist eine Eigenschaft von Raft, nicht der Betriebsart. Der Pod ist der aktive (und einzige) Knoten.

Ohne Argument fragt `bao operator unseal` den Key interaktiv ab und liest ihn ohne Echo. Ein Key als Argument stünde in der Shell-History und wäre über `/proc` für andere Prozesse sichtbar.

Nach dem Unseal wechselt der Pod auf `READY 1/1`, und `service_registration` setzt `openbao-sealed: "false"` als Label – ab jetzt hat der Service ein Backend.

Was nach jedem Neustart passiert

Der Pod stirbt, wird verschoben, das Node wird gepatcht: OpenBao startet **versiegelt**. Das ist der bewusst akzeptierte Preis der Shamir-Entscheidung. Solange nichts davon abhängt, ist es nur lästig. Sobald der External Secrets Operator daran hängt (Nº 2), ist jeder Neustart ein Ausfall für alles, was Secrets erneuern muss. An diesem Punkt gehört Auto-Unseal (Transit-Engine auf dem VM-OpenBao, oder ein Cloud-KMS) auf die Liste.

Der erste Login

```
/ $ bao login
Token (will be hidden):
```

Der Root-Token. Ab hier ist die Shell `root` in OpenBao – und `bao login` schreibt das Token nach `~/.vault-token` im Container. Dazu gleich mehr in Teil V; erst einmal ist das der Zustand, in dem man die Konfiguration vornimmt.

Teil III – Kubernetes-Auth

Wie es funktioniert

Jeder Pod in Kubernetes hat einen Ausweis: das Token seines ServiceAccounts. Kubernetes legt es beim Start als Datei in den Pod (`/var/run/secrets/kubernetes.io/serviceaccount/token`), erneuert es regelmäßig, und es sagt aus, welcher ServiceAccount in welchem Namespace der Pod ist. Der Pod hat es, ohne dass jemand es ihm geben musste.

Genau diesen Ausweis benutzt der Login. Vier Schritte:

1. Der Pod schickt sein Token an OpenBao (`auth/kubernetes/login`) und nennt eine Rolle, zum Beispiel `demo`.
2. OpenBao kann das Token nicht selbst prüfen – es hat es nicht ausgestellt. Also fragt es den, der es kann: den Kubernetes-API-Server. Dafür gibt es eine eigene API, `TokenReview`: „Ist dieses Token echt, und wem gehört es?“
3. Der API-Server antwortet: echt, ServiceAccount `demo` im Namespace `demo`.
4. OpenBao vergleicht das mit der Rolle. Passen ServiceAccount und Namespace zu dem, was in der Rolle steht, stellt es einen OpenBao-Token aus – mit den Policies, die die Rolle nennt. Passen sie nicht, gibt es `403`.

Der Punkt daran: Nirgends musste ein Passwort oder ein Schlüssel verteilt werden. Der Pod hatte seinen Ausweis schon, und OpenBao braucht für die Rückfrage beim API-Server nur, was es als Pod selbst hat – sein eigenes ServiceAccount-Token und das CA-Zertifikat des Clusters.

Aktivieren und konfigurieren

```
/ $ bao auth enable kubernetes
/ $ bao write auth/kubernetes/config \
    kubernetes_host="https://kubernetes.default.svc:443"
```

Mehr nicht. Weil OpenBao selbst im Cluster läuft, nimmt es für den `TokenReview` sein eigenes ServiceAccount-Token und die CA aus `/var/run/secrets/kubernetes.io/serviceaccount/`. Die Berechtigung dafür bringt das Chart mit: Es rendert ein `ClusterRoleBinding` des ServiceAccounts `openbao` auf die eingebaute `ClusterRole` `system:auth-delegator` – das ist exakt das Recht, `TokenReviews` zu stellen, und sonst nichts. Läuft OpenBao **außerhalb** des Clusters, müssen `kubernetes_ca_cert` und `token_reviewer_jwt` explizit gesetzt werden – und das Reviewer-Token braucht die `ClusterRole` `system:auth-delegator`.

Eine Rolle für einen Consumer

Eine Rolle bindet einen ServiceAccount an Policies:

```
/ $ bao write auth/kubernetes/role/demo \  
    bound_service_account_names=demo \  
    bound_service_account_namespaces=demo \  
    token_policies=default \  
    token_ttl=1h
```

```
/ $ bao read auth/kubernetes/role/demo  
Key                                Value  
---                                -  
alias_name_source                  serviceaccount_uid  
bound_service_account_names        [demo]  
bound_service_account_namespaces   [demo]  
token_policies                     [default]  
token_ttl                           1h
```

alias_name_source: serviceaccount_uid bedeutet: Die Identität in OpenBao hängt an der UID des ServiceAccounts. Wird der ServiceAccount gelöscht und neu angelegt, ist es aus OpenBaos Sicht eine neue Identität – der alte Alias bleibt als Leiche im Identity-Store, bis man ihn entfernt.

default ist die Policy, die jedes Token ohnehin bekommt – sie erlaubt kaum mehr als den Blick auf das eigene Token. Für diesen Leitfaden reicht das: Die Rolle sagt *wer* darf sich anmelden und *welche* Policy er bekommt. Was ein echter Consumer lesen darf, legt man fest, wenn es ihn gibt.

Prüfen, ohne ESO

Der schnellste Test ist ein Pod mit dem ServiceAccount, der den Login von Hand macht:

```
kubectl create namespace demo  
kubectl create serviceaccount demo -n demo  
kubectl run -n demo --rm -it --restart=Never \  
    --overrides='{\"spec\":{\"serviceAccountName\":\"demo\"}}' \  
    --image=curlimages/curl:8.11.1 probe -- sh
```

Der Pod tut so, als wäre er die spätere Anwendung. Entscheidend ist `--overrides`: `kubectl run` hat keinen Flag mehr, um den ServiceAccount zu setzen, also wird das Feld als JSON-Fragment über das generierte Manifest gelegt. Ohne diese Zeile lief der Pod als default – und die Rolle demo würde ihn ablehnen, egal wie richtig alles andere ist. `--restart=Never` sorgt dafür, dass wirklich nur ein Pod entsteht und kein Deployment; `--rm` löscht ihn nach exit; `-- sh` ersetzt den Einstiegspunkt des Images durch eine Shell. Das curl-Image ist bewusst gepinnt, damit der Test in einem Jahr noch dasselbe tut.

Im Pod:

```
JWT=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)  
curl -s http://openbao.openbao.svc:8200/v1/auth/kubernetes/login \  
-d '{\"role\":\"demo\", \"jwt\":\"$JWT\"}' | head -c 400
```

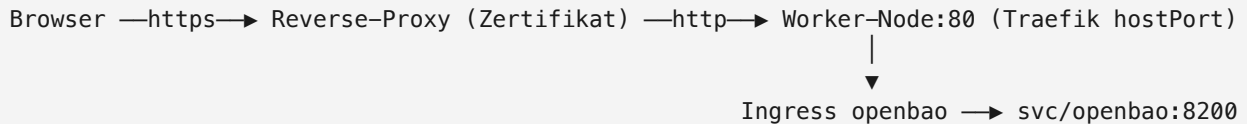
Das ist der Login, den ESO später automatisch macht – hier zu Fuß. Die erste Zeile liest das Token, das Kubernetes jedem Pod für seinen ServiceAccount einhängt: ein signiertes JWT, das Namespace und Name des ServiceAccounts enthält. Die zweite schickt es an die Auth-Methode – ohne eigenes

Token, denn der Login-Pfad ist unauthentifiziert; die Antwort ist das Token. `openbao.openbao.svc` ist der Cluster-DNS-Name des Service (Name.Namespace.svc), Port 8200 der Listener; der Weg über den Ingress ist für Pods unnötig. Die Escapes im `-d-Body` sind nötig, weil nur doppelte Anführungszeichen `$JWT` expandieren. `head -c 400` schneidet die Antwort ab, damit nicht das ganze Token samt accessor im Scrollback landet.

Ein `client_token` in der Antwort heißt: Auth-Methode, TokenReview und Rolle funktionieren. Eine `403 permission denied` heißt fast immer: Namespace oder Name des ServiceAccounts passen nicht zur Rolle – nicht, dass die Policy etwas verbietet, denn die kommt erst nach dem Login ins Spiel.

Teil IV – Der Zugang von außen

Der Weg



Traefik läuft in RKE2 als DaemonSet mit hostPort 80/443 – aber nur auf den Workern, weil die Control-Plane-Nodes getaintet sind und das DaemonSet die Taints nicht toleriert. Das Ziel des Reverse-Proxys ist darum die IP eines Workers. Fällt dieser Worker aus, ist die UI nicht erreichbar, obwohl OpenBao weiterläuft. Der Proxy selbst – Nginx Proxy Manager, DuckDNS, Let’s Encrypt – ist in Anhang C beschrieben.

Eine floatende VIP (MetalLB) würde das lösen. Sie wurde bewusst nicht installiert: Die Verfügbarkeit der UI ist in einem Lab keinen weiteren Controller wert – die Workloads im Cluster erreichen OpenBao über svc/openbao, und das ist von keinem Worker abhängig.

Der bao-CLI von außen

```
export BAO_ADDR=https://bao.example.internal
bao status
bao login    # Token
```

Zwei Fallen, die beim Debuggen Zeit kosten:

- Ein Terminal mit BAO_ADDR auf den Ingress, aber ohne Token: **jeder** Aufruf antwortet 403 permission denied – auch `bao token capabilities`. Das sieht aus wie ein Policy-Problem und ist ein fehlendes Login. `bao token lookup` zuerst.
- `bao login` schreibt nach `~/.vault-token` – denselben Pfad, den die Vault-CLI benutzt. Wer parallel ein Vault oder einen zweiten OpenBao betreibt, überschreibt beim Login des einen das Token des anderen. Ein rätselhaftes 403 ist dann meist ein Token vom anderen System.

Die UI

`https://bao.example.internal/ui/` – Login mit dem Root-Token oder, nach der Härtung, per Username. Die UI ist praktisch für den Überblick über Mounts, Policies und Leases; für alles Reproduzierbare ist die CLI besser, weil ihre Aufrufe in ein Skript oder nach OpenTofu wandern können.

Teil V – Der Ist-Zustand, ehrlich

Das ist der Punkt, an dem dieser Leitfaden hinter dem VM-Aufbau (N° 3) zurückbleibt. Auf der VM ist die Bootstrap-Sequenz vollständig durchgeführt und verifiziert. Im Cluster ist sie es **nicht**. So sieht es aus:

```
/ $ bao audit list
No audit devices are enabled.

/ $ bao auth list
Path          Type          Description
----          -
kubernetes/   kubernetes    n/a
token/        token         token based credentials

/ $ bao policy list
default
root

/ $ bao token lookup | grep -E 'display_name|policies'
display_name   root
policies       [root]

/ $ ls -la ~/.vault-token
-rw----- 1 openbao openbao /home/openbao/.vault-token
```

Vier Befunde:

- 1. Kein Audit-Device.** Niemand kann beantworten, wer wann welchen Pfad gelesen hat. Bei einem Secrets-Store ist das der Hauptgrund, ihn überhaupt zu betreiben.
- 2. Der Root-Token ist der einzige administrative Zugang.** Es gibt keinen userpass, keine Admin-Policy. Der Root-Token läuft nie ab und kann nicht eingeschränkt werden.
- 3. Der Root-Token liegt als Datei im Pod.** `bao login` hat ihn nach `/home/openbao/.vault-token` geschrieben. Jeder mit `kubectl exec` auf den Pod – also jeder mit ausreichenden RBAC-Rechten im Namespace `openbao` – ist damit root in OpenBao. Die Datei lebt im Container-Dateisystem und verschwindet beim Neustart; bis dahin ist sie da.
- 4. Keine Snapshots.** Das PVC ist die einzige Kopie der Raft-Daten. Longhorn repliziert das Volume zwar über Nodes, aber das schützt gegen einen Plattenausfall, nicht gegen ein versehentliches `helm uninstall` mit gelöschtem PVC, ein kaputtes Upgrade oder einen Fehlgriff mit `bao delete`.

Für eine Demo tragbar, für alles andere nicht. Teil VI beschreibt, was zu tun ist – die Kommandos stammen aus dem VM-Runbook (N° 3) und wurden für diese Note auf einem kind-Cluster (siehe „Der Cluster“) durchgespielt. Zwei davon gehen auf OpenBao 2.6 nicht mehr so wie auf der VM; Teil VI zeigt, welche und warum.

Teil VI – Härtung: was als Nächstes kommt

Die Reihenfolge ist nicht beliebig. Audit zuerst, damit alles Folgende protokolliert ist. Root-Token zuletzt, und erst, wenn der Ersatzzugang **bewiesen** ist.

1. Token-Datei aus dem Pod entfernen

Sofort, unabhängig vom Rest:

```
/ $ rm ~/.vault-token
```

Für die weitere Arbeit das Token nur noch in der Umgebungsvariable der laufenden Shell halten:

```
/ $ read -rs BAO_TOKEN; export BAO_TOKEN
```

`read -rs` liest ohne Echo und ohne History-Eintrag. Beim Schließen der Shell ist das Token weg.

2. Audit-Device

Der naheliegende Befehl – auf der VM in N° 3 noch der richtige – geht auf 2.6.2 nicht mehr:

```
$ bao audit enable file file_path=/openbao/audit/audit.log
Error enabling audit device: Error making API request.
Code: 400. Errors:

* cannot enable audit device via API; use declarative, config-based audit device management instead
```

Seit OpenBao 2.3.2 ist das Anlegen von Audit-Devices über die API standardmäßig abgeschaltet (`unsafe_allow_api_audit_creation = false`). Der Grund ist gut: Ein file-Device schreibt an einen beliebigen Pfad, ein socket-Device an einen beliebigen Socket – wer einen Admin-Token erbeutet, konnte damit den Server als Schreibwerkzeug benutzen. Seit OpenBao 2.4.0 gehören Audit-Devices darum als `audit-Block` in die Server-Konfiguration – dieselbe HCL-Datei, in der schon `listener` und `storage` stehen. Auf der VM wäre das `/etc/openbao/openbao.hcl`; im Cluster schreibt das Helm-Chart diese Datei aus dem Value `server.standalone.config` in eine `ConfigMap` und mountet sie unter `/openbao/config`. Beide Versionsnummern sind also OpenBao-Versionen, nicht die des Charts (0.29.4) oder von RKE2. Zwei Ergänzungen in den Values – der Block und das Volume, auf das er schreibt (im Repository als `k8s/values-hardened.yaml`, das zusätzlich zu `k8s/values.yaml` übergeben wird):

```

server:
  standalone:
    config: |
      # ... ui, listener und storage wie in Teil I ...

      audit "file" "audit" {
        options = {
          file_path = "/openbao/audit/audit.log"
        }
      }

  auditStorage:
    enabled: true
    size: 2Gi
    storageClass: longhorn

```

/openbao/audit ist der Pfad, den das Chart mit auditStorage als eigenes PVC bereitstellt – ohne das Volume liegt das Log im Container und ist nach dem Neustart weg. Ein Device aus der Konfiguration lässt sich per API weder ändern noch löschen; verschwindet der Block, verschwindet das Device beim nächsten Neustart.

Die zweite Hürde: `helm upgrade` mit diesen Values **schlägt fehl**.

```

Error: UPGRADE FAILED: StatefulSet.apps "openbao" is invalid: spec: Forbidden:
updates to statefulset spec for fields other than 'replicas', 'ordinals',
'template', 'updateStrategy', 'revisionHistoryLimit',
'persistentVolumeClaimRetentionPolicy' and 'minReadySeconds' are forbidden

```

Ein zweites Volume ist ein zweites `volumeClaimTemplate`, und das ist an einem bestehenden `StatefulSet` unveränderlich. Der Weg führt über ein neues `StatefulSet` – und genau dafür steht `whenDeleted: Retain` in Teil I:

```

kubectl delete sts -n openbao openbao      # PVC data-openbao-0 bleibt (Retain)
helm upgrade --install openbao openbao/openbao --version 0.29.4 \
  --namespace openbao --values k8s/values.yaml --values k8s/values-hardened.yaml
kubectl exec -it -n openbao openbao-0 -- bao operator unseal    # x3

```

```

$ kubectl get pvc -n openbao
NAME                STATUS    CAPACITY
audit-openbao-0     Bound    2Gi
data-openbao-0      Bound    10Gi

$ bao audit list
Path      Type      Description
----      -
audit/    file      n/a

```

Das Chart legt das `StatefulSet` neu an, der Pod findet sein Daten-PVC wieder, und OpenBao aktiviert das Device beim Start – im Log steht `core: enabled audit backend: path=audit/ type=file`. Wer neu aufsetzt, übergibt beide Values-Dateien von Anfang an und spart sich das Löschen; `k8s/values.yaml` allein zeigt bewusst den Stand *vor* der Härtung.

Zwei Dinge, die man wissen muss: Kann OpenBao nicht ins Audit-Log schreiben, **verweigert es Anfragen**. Ein volles Volume nimmt den Secrets-Store vom Netz. Das ist ein Sicherheitsfeature und ohne Rotation eine Zeitbombe – auf der VM übernimmt logrotate das; im Cluster braucht es einen Sidecar oder einen zweiten Audit-Device (syslog, socket zu einem Log-Collector), der zur primären Senke wird. Und: Ab jetzt zeigen alle Einträge die IP von Traefik statt des echten Clients, solange `x_forwarded_for_authorized_addrs` im Listener nicht gesetzt ist.

3. Admin-Policy und userpass

Schritt 5 widerruft den Root-Token. Vorher braucht es einen anderen Weg hinein – einen, der abläuft, an eine Person gebunden ist und im Audit-Log mit Namen erscheint. Dafür zwei Dinge: eine **Policy** `admin`, die sagt, was dieser Zugang darf, und ein **userpass-Login** `admin`, der beim Anmelden ein Token mit genau dieser Policy bekommt (die Policy liegt auch als `openbao/admin.hcl` im Repository):

```
/ $ bao policy write admin - <<'EOF'
path "sys/*" {
  capabilities = ["create", "read", "update", "delete", "list", "sudo"]
}
path "auth/*" {
  capabilities = ["create", "read", "update", "delete", "list", "sudo"]
}
path "identity/*" {
  capabilities = ["create", "read", "update", "delete", "list"]
}
EOF

/ $ bao auth enable userpass
/ $ bao write auth/userpass/users/admin \
  password="$(read -rs -p 'Passwort: ' p; echo "$p")" \
  token_policies=admin token_ttl=1h token_max_ttl=8h
```

Drei Zeilen, und sie sind absichtlich breit: `sys/*` ist die Verwaltung – Mounts, Auth-Methoden, Policies, Audit, Snapshots und der Weg zu einem neuen Root-Token (`sys/generate-root-token/*`, Schritt 4). `auth/*` und `identity/*` sind Benutzer, Rollen und Identitäten. Was fehlt, fehlt mit Absicht: Daten-Pfade. Der Admin verwaltet OpenBao, er liest keine Secrets. Eine lange Liste einzelner `sys/-`Pfade wäre kein Gewinn – wer Policies schreiben darf, kann sich alles Weitere selbst geben. Der Gewinn gegenüber dem Root-Token ist nicht weniger Macht, sondern **Ablauf, Zurechenbarkeit und ein Passwort, das sich rotieren lässt**.

Das Passwort gehört in den Passwort-Manager, **bevor** Schritt 5 kommt. Unter OpenBao ist es Recovery-Material, nicht Komfort: Seit 2.5.3 sind die unauthentifzierten `sys/generate-root/*`-Endpunkte standardmäßig aus, und seit 2.6.0 nutzt `bao operator generate-root` die authentifzierten `sys/generate-root-token`-Endpunkte (eine ältere CLI spricht noch die alten an und bekommt 405 – siehe Teil VII). Drei Unseal-Keys allein sind also **kein Weg zurück** – es braucht zusätzlich einen Login, dessen Policy `sys/*` enthält.

4. Den Ersatzzugang beweisen – nicht überspringen

```
# -token-only: sonst überschreibt 'bao login' das laufende Root-Token
# in ~/.vault-token, BEVOR bewiesen ist, dass der neue Zugang funktioniert.
/ $ ADMIN_TOKEN="$(bao login -method=userpass -token-only username=admin)"

/ $ BAO_TOKEN="$ADMIN_TOKEN" bao token lookup | grep policies
/ $ BAO_TOKEN="$ADMIN_TOKEN" bao policy list

# Der Break-Glass-Pfad selbst: -init startet eine Root-Generierung,
# -cancel bricht sie ab. Das beweist die Fähigkeit, ohne einen Root-Token
# zu erzeugen, den man dann verwahren müsste.
/ $ BAO_TOKEN="$ADMIN_TOKEN" bao operator generate-root -init
/ $ BAO_TOKEN="$ADMIN_TOKEN" bao operator generate-root -cancel

/ $ unset ADMIN_TOKEN
```

Erst wenn admin in den Policies steht, `policy list` funktioniert **und** das `generate-root -init/-cancel`-Paar durchläuft, ist der Ersatzzugang bewiesen. Schlägt irgendetwas davon fehl: Schritt 5 **nicht** ausführen.

5. Root-Token widerrufen

```
/ $ bao token lookup | grep policies      # [root]
/ $ bao token revoke -self
/ $ bao token lookup                      # muss jetzt fehlschlagen
```

Ab hier ist userpass der Weg hinein. Break Glass, falls der Zugang verloren geht:

```
bao login -method=userpass username=admin
bao operator generate-root -init
bao operator generate-root      # 3x mit Unseal-Keys
```

6. Ein erster Snapshot

Befund 4 aus Teil V: Das PVC ist die einzige Kopie der Raft-Daten. Bevor der Root-Token weg ist, gehört ein Snapshot auf einen Rechner außerhalb des Clusters – zwei Kommandos:

```
kubectl exec -n openbao openbao-0 -- \
  sh -c 'BAO_TOKEN=... bao operator raft snapshot save /tmp/bao.snap' && \
kubectl cp openbao/openbao-0:/tmp/bao.snap ./openbao-$(date +%Y%m%dT%H%M).snap
```

Und die Prüfung, die OpenBao selbst nicht anbietet (`raft snapshot inspect` gibt es nur bei Vault): Das Archiv ist ein gzipped tar mit vier Einträgen –

```
tar -tzf openbao-*.snap
tar -xz0f openbao-*.snap SHA256SUMS
tar -xz0f openbao-*.snap state.bin | sha256sum      # muss zur Zeile oben passen
```

```
meta.json
state.bin
SHA256SUMS
SHA256SUMS.sealed
30415d1f6fc46454441004a9c4e01eaf764c53a0051cac3fcc93eae288223249 meta.json
ffcf434b4ad464420a558a883706f929306f7df966b948bc41dd2268a87d3047 state.bin
ffcf434b4ad464420a558a883706f929306f7df966b948bc41dd2268a87d3047 -
```

Das ist ein Snapshot, kein Backup: Er liegt auf deinem Rechner, niemand zieht ihn regelmäßig, und niemand hat den Restore geprobt. Der CronJob mit eigener Identität, die Ablage im Objekt-Storage und die Restore-Probe sind N° 7.

Damit endet die Härtung von Hand. Alles aus Teil III und VI ist Zustand, der bei einem Neuaufbau weg ist; ihn als Code zu fassen – Policies, Auth-Methoden, Rollen – ist N° 9.

Teil VII – Was schiefgehen kann

Pod bleibt 0/1 Running

Erwartet, solange OpenBao versiegelt oder nicht initialisiert ist. `kubectl exec ... bao status` zeigt, welcher Fall vorliegt. Erst wenn `Sealed false` und der Pod trotzdem nicht Ready wird, ist etwas kaputt – dann die Probe im StatefulSet ansehen (`kubectl describe pod`).

Log voller `service_registration ... 403`

Alle fünf Sekunden eine Warnung: `service_registration "kubernetes"` ist konfiguriert, aber der ServiceAccount darf den eigenen Pod nicht patchen. Im Standalone-Modus rendert das Chart die nötige Role nicht. Den Block aus der Server-Config entfernen (siehe Teil I) oder eine Role mit `get`, `update`, `patch` auf pods und ein RoleBinding auf den ServiceAccount `openbao` anlegen.

`helm upgrade` ändert nichts

`OnDelete`. Pod löschen, `unseal`. Siehe Teil I.

`403 permission denied` bei allem

Kein Token, abgelaufenes Token, oder Token vom falschen System. `bao token lookup` zuerst. Erst wenn das funktioniert und ein bestimmter Pfad trotzdem 403 gibt, ist es die Policy – `bao token capabilities <pfad>` zeigt deny.

Kubernetes-Login schlägt fehl

Meldung	Ursache
<code>service account name not authorized</code>	<code>bound_service_account_names</code> passt nicht
<code>namespace not authorized</code>	<code>bound_service_account_namespaces</code> passt nicht
Post "https://kubernetes.default.svc:443/.../tokenreviews": ...	OpenBao erreicht den API-Server nicht, oder sein eigenes Token darf keinen TokenReview stellen
<code>permission denied</code> erst beim Lesen eines Pfads	Login war erfolgreich; die Policy der Rolle erlaubt den Pfad nicht

OpenBao nimmt keine Anfragen mehr an, Log sagt audit

Das Audit-Volume ist voll oder nicht beschreibbar. OpenBao verweigert dann absichtlich. Volume vergrößern oder Log rotieren – und danach einen zweiten Audit-Device einrichten, damit das nicht wieder passiert.

generate-root antwortet 405 unsupported operation

Die CLI ist älter als 2.6.0 und spricht noch `sys/generate-root/attempt` an, das der Server seit 2.5.3 nicht mehr bedient. Die CLI im Pod passt immer zum Server: `kubectl exec -it -n openbao openbao-0 -- sh`.

Nach Node-Wartung: versiegelt

Erwartet. `kubectl exec -it -n openbao openbao-0 -- bao operator unseal`, dreimal. Wenn das zu oft passiert: Auto-Unseal, siehe Teil II.

Anhang A – Alle Kommandos

Dieselbe Liste liegt als `commands.sh` im Repository.

```
# — Helm —————
helm repo add openbao https://openbao.github.io/openbao-helm
helm upgrade --install openbao openbao/openbao --version 0.29.4 \
  -n openbao --create-namespace -f k8s/values.yaml

# — init / unseal (Shell im Pod, normales Terminal) —————
kubectl exec -it -n openbao openbao-0 -- sh
bao operator init -key-shares=5 -key-threshold=3      # Ausgabe -> Passwort-Manager
bao operator unseal                                  # x3
bao status
read -rs BAO_TOKEN; export BAO_TOKEN                # kein 'bao login' im Pod

# — Kubernetes-Auth —————
bao auth enable kubernetes
bao write auth/kubernetes/config kubernetes_host="https://kubernetes.default.svc:443"
bao write auth/kubernetes/role/demo \
  bound_service_account_names=demo bound_service_account_namespaces=demo \
  token_policies=default token_ttl=1h

# — Härtung (Reihenfolge einhalten) —————
# Audit: k8s/values-hardened.yaml (audit-Block + auditStorage), dann:
kubectl delete sts -n openbao openbao                # PVC bleibt (Retain)
helm upgrade --install openbao openbao/openbao --version 0.29.4 \
  -n openbao -f k8s/values.yaml -f k8s/values-hardened.yaml
bao operator unseal                                  # x3, dann: bao audit list
bao policy write admin openbao/admin.hcl
bao auth enable userpass
bao write auth/userpass/users/admin password=... \
  token_policies=admin token_ttl=1h token_max_ttl=8h
ADMIN_TOKEN="$(bao login -method=userpass -token-only username=admin)"
BAO_TOKEN="$ADMIN_TOKEN" bao operator generate-root -init
BAO_TOKEN="$ADMIN_TOKEN" bao operator generate-root -cancel
bao token revoke -self                               # erst nach bestandenem Test

# — Snapshot von Hand —————
kubectl exec -n openbao openbao-0 -- \
  sh -c 'bao operator raft snapshot save /tmp/bao.snap'
kubectl cp openbao/openbao-0:/tmp/bao.snap ./openbao-$(date +%Y%m%dT%H%M).snap
tar -xz0f openbao-*.snap state.bin | sha256sum
```

Anhang B – Glossar

Seal / Unseal – OpenBao startet mit verschlüsseltem Storage und ohne den Schlüssel dazu. Unseal setzt den Master-Key aus Shamir-Anteilen zusammen; bis dahin beantwortet OpenBao nur status.

Shamir 5/3 – Der Master-Key ist in fünf Anteile zerlegt, drei beliebige davon rekonstruieren ihn.

Raft – Integriertes Storage-Backend mit Konsens-Protokoll. Auch mit einem Knoten sinnvoll, weil nur Raft Snapshots kann.

Root-Token – Der Token aus init. Unbegrenzt gültig, nicht einschränkbar. Wird nach dem Bootstrap widerrufen und bei Bedarf per generate-root neu erzeugt.

Auth-Methode – Wie ein Client seine Identität nachweist. kubernetes prüft ServiceAccount-Tokens per TokenReview; userpass prüft Passwörter.

Rolle (Auth) – Bindet Identitätsmerkmale (ServiceAccount, Namespace) an Policies und Token-Eigenschaften.

Policy – Erlaubte Pfade und Capabilities. Default: alles verboten.

Audit-Device – Protokolliert jede Anfrage und Antwort (mit gehashten Secrets). Kann OpenBao nicht schreiben, verweigert es Anfragen. Wird seit 2.4 in der Server-Konfiguration definiert, nicht per API.

TokenReview – Kubernetes-API, mit der ein Dritter prüfen lässt, ob ein ServiceAccount-Token gültig ist und zu wem es gehört.

Break Glass – Der Weg zu einem neuen Root-Token, wenn der alte weg ist. Unter OpenBao braucht er Unseal-Keys **und** einen Login mit sys/generate-root-token/*.

OnDelete – Update-Strategie des StatefulSets: neue Konfiguration wird erst wirksam, wenn der Pod von Hand gelöscht wird.

Anhang C – Der Reverse-Proxy

Teil IV setzt einen Reverse-Proxy vor dem Cluster voraus, der das TLS-Zertifikat hält und `bao.example.internal` an einen Worker weiterreicht. Hier ist es der **Nginx Proxy Manager** (NPM) in einer eigenen kleinen VM, mit einem Namen von **DuckDNS** und einem Zertifikat von Let's Encrypt. Wer schon einen Proxy hat – Caddy, Traefik auf dem Host, ein nginx von Hand – braucht diesen Anhang nicht.

Warum DuckDNS, und warum DNS-01

Der Cluster ist von außen nicht erreichbar, und das soll so bleiben. Ein Zertifikat von Let's Encrypt setzt normalerweise voraus, dass Let's Encrypt den Host über Port 80 erreicht (HTTP-01) – das fällt weg. Die Alternative ist die DNS-01-Challenge: Let's Encrypt prüft einen TXT-Record in der DNS-Zone, und dafür muss der Proxy die Zone schreiben dürfen. DuckDNS ist ein kostenloser dynamischer DNS-Dienst, dessen API genau das kann, und NPM bringt ihn als Provider mit. Das Ergebnis: ein echtes Zertifikat für einen Namen, der auf eine private IP zeigt, ohne dass ein Port nach außen offen ist.

Zwei Dinge, die man dazu wissen muss. Erstens: Der Name ist öffentlich. `<name>.duckdns.org` steht im Certificate-Transparency-Log, sobald das Zertifikat ausgestellt ist; die IP dahinter ist privat, der Name nicht. Zweitens: DuckDNS löst auch alles *unterhalb* der eigenen Subdomain auf – `bao.<name>.duckdns.org` zeigt auf dieselbe IP wie `<name>.duckdns.org`. Ein Wildcard-Zertifikat deckt darum alle Dienste ab, die später hinter dem Proxy landen.

Wer nur schnell testen will: `bao.10-0-0-20.sslip.io` löst ohne jede Einrichtung nach `10.0.0.20` auf (nip.io ebenso) – die IP im Namen ist die des Workers, auf dem Traefik Port 80 hält, also das `<worker-ip>` aus Teil I; bei einer anderen Adresse entsprechend ersetzen. DNS allein reicht aber nicht: Traefik routet nach dem Host-Header, und der Ingress kennt nur den Namen aus `server.ingress.hosts`. Der sslip-Name muss also dort hinein – derselbe `helm upgrade` wie in Teil I, nur mit diesem Host; ein Neustart des Pods ist dafür nicht nötig. Fehlt das, antwortet Traefik mit `404 page not found`, obwohl DNS und Verbindung stimmen. Ein Let's-Encrypt-Zertifikat gibt es für den Namen nicht, weil die IP privat ist und die Zone einem nicht gehört – für den ersten Blick auf die UI tut es dann auch HTTP.

1. DuckDNS

Auf duckdns.org anmelden, eine Subdomain anlegen (`<name>`), als IP die **private** Adresse der Proxy-VM eintragen. Auf der Seite steht der Token – der gehört in den Passwort-Manager, NPM braucht ihn gleich. Die IP lässt sich auch per API setzen:

```
curl "https://www.duckdns.org/update" \
  --data-urlencode "domains=<name>" \
  --data-urlencode "token=<duckdns-token>" \
  --data-urlencode "ip=<ip-der-proxy-vm>"
```

DuckDNS antwortet mit OK. Prüfen: `dig +short bao.<name>.duckdns.org` muss die IP der Proxy-VM liefern.

2. Die Proxy-VM

Eine kleine VM mit Docker – 1 vCPU und 1 GB reichen. NPM läuft als ein Container, Konfiguration und Zertifikate liegen in zwei Verzeichnissen daneben:

```
# docker-compose.yml
services:
  npm:
    image: jc21/nginx-proxy-manager:latest
    restart: unless-stopped
    ports:
      - "80:80"      # HTTP, wird auf HTTPS umgeleitet
      - "443:443"    # HTTPS
      - "81:81"      # Admin-UI, nur im LAN
    volumes:
      - ./data:/data
      - ./letsencrypt:/etc/letsencrypt
```

```
docker compose up -d
```

Die Admin-UI ist unter `http://<ip-der-proxy-vm>:81` erreichbar. Beim ersten Login (`admin@example.com / changeme`) verlangt NPM sofort eine neue Adresse und ein neues Passwort. Port 81 gehört nicht hinter DuckDNS und nicht nach außen – die Admin-UI ist der Schlüssel zu allem, was der Proxy weiterleitet.

3. Das Zertifikat

In der Admin-UI: *SSL Certificates* → *Add SSL Certificate* → *Let's Encrypt*.

- Domain Names: `<name>.duckdns.org` und `*.<name>.duckdns.org`
- *Use a DNS Challenge* einschalten, Provider **DuckDNS**
- Credentials: `dns_duckdns_token=<duckdns-token>`
- Propagation Seconds: 60 – DuckDNS braucht einen Moment, bis der TXT-Record sichtbar ist

Nach ein bis zwei Minuten steht das Zertifikat in der Liste. NPM erneuert es selbst, solange der Token gültig bleibt.

4. Der Proxy Host

Hosts → *Proxy Hosts* → *Add Proxy Host*:

- Domain Names: `bao.<name>.duckdns.org`
- Scheme `http`, Forward Hostname die IP eines **Workers**, Forward Port `80` – warum ein Worker und nicht die Control-Plane, steht in Teil IV
- Reiter *SSL*: das Wildcard-Zertifikat auswählen, *Force SSL* an

NPM reicht den Host-Header unverändert weiter, und genau daran erkennt Traefik, welcher Ingress gemeint ist. Der Name muss darum an drei Stellen übereinstimmen: hier im Proxy Host, in `server.ingress.hosts` der Values (Teil I) und in `BA0_ADDR` (Teil IV). `bao.example.internal` in diesem

Leitfaden ist also überall als `bao.<name>.duckdns.org` zu lesen.

Prüfen

```
H=https://bao.<name>.duckdns.org
curl -s -o /dev/null -w '%{http_code}\n' $H/ui/
curl -s -o /dev/null -w '%{http_code}\n' $H/v1/sys/health
```

Die UI antwortet mit 200 – ohne -k, das Zertifikat ist echt. sys/health antwortet vor init mit 501 und versiegelt mit 503: Das ist ein Proxy, der funktioniert, und ein OpenBao, das noch auf Teil II wartet.

Was hier fehlt, ehrlich

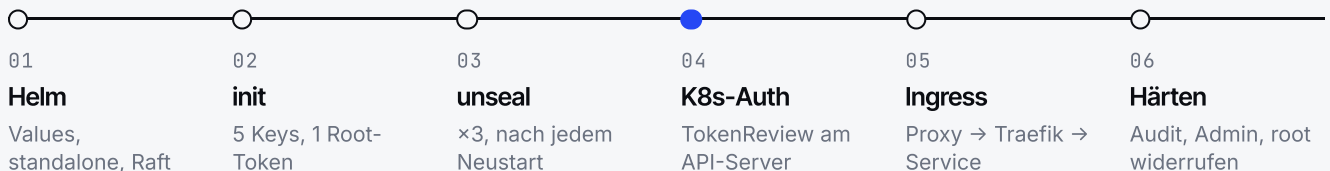
Der DuckDNS-Token liegt unverschlüsselt in `./data` auf der Proxy-VM – wer die VM hat, hat die Zone. Die Admin-UI hat Benutzernamen und Passwörter, sonst nichts. Und NPM wird nicht automatisch aktualisiert. Für ein Lab tragbar; in allem anderen wäre der Proxy der erste Kandidat für dieselbe Härtung, die Teil VI für OpenBao beschreibt.

Der Pod läuft. Er ist versiegelt. Und genau das ist richtig.

Dieser Leitfaden dokumentiert den Aufbau eines OpenBao im Kubernetes-Cluster, so wie er tatsächlich passiert ist: Helm-Values mit Begründung, init und unseal in einer Pod-Shell, Kubernetes-Auth mit TokenReview, der Weg von außen durch Proxy und Traefik.

Und den Zustand danach, ehrlich: kein Audit-Device, der Root-Token als Datei im Pod, ein service_registration-Block, der 22 Stunden lang alle fünf Sekunden mit 403 scheiterte. Dazu die Härtung, die daraus folgt – in der Reihenfolge, die einen Lockout ausschließt.

```
-- Die eine Zeile, die Backups überhaupt möglich macht
storage "raft" {
  path = "/openbao/data"
}
```



kubernetes	RKE2 v1.36 · Traefik · Longhorn
openbao	2.6.2 · chart 0.29.4 · raft
helm	3.16 · standalone
auth	kubernetes · TokenReview
storage	10Gi PVC · Retain

36 Seiten · 7 Teile · 3 Anhänge
Vertiefung: **Vault in Practice**
leanpub.com/vault-in-practice
Gesetzt in Inter Tight, Inter und JetBrains Mono.
Alle Ausgaben sind echte Terminal-Ausgaben.
CC BY 4.0 · Weitergabe erlaubt · ohne Gewähr